PYTHON ACTIVITY-BASED LEARNING

Complete Report: 10 Selected Case Studies

Student Detail	Information
Name	Bhanuprakash Seela
Roll Number	26001495
Branch	CSD
Section	2

This report contains the problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output screenshots, and conclusion for each selected case study.

Note: Fees and thresholds are illustrative where the question sheet does not specify exact values. Confirm any required rules with your instructor.

Contents

1. 1. University Scholarship Allocation
2. 2. Attendance Recovery Computation
3. 3. ATM Transaction Sequence Simulation
4. 4. Smart Parking Fee Management
5. 5. Student Performance Trend Analysis
6. 6. Fuel Consumption and Route Comparison
7. 7. Multi-Product Shopping Budget Constraint
8. 8. Advanced Numeric Pattern Generator
9. 9. Password Policy Validation Engine
10. 10. Multi-Stage Examination Eligibility System

Implementation note

The programs use beginner-level Python: variables, input/output, arithmetic, if/else, loops, strings, and lists. Output images are sample console runs.

1. University Scholarship Allocation

Problem Statement

Read student names, marks in three subjects, attendance, and family income. Check whether each student meets all scholarship eligibility conditions.

Objectives

• Read applicant information.

• Calculate average marks.

• Check subject marks, attendance, and income conditions.

• Print an eligibility decision.

Assumptions

• Three subjects are used.

• Average marks ≥ 75; each subject mark ≥ 60.

• Attendance ≥ 75%.

• Annual family income ≤ ■250,000.

• This version checks eligibility but does not rank students.

Algorithm

1. Start and read the number of students.

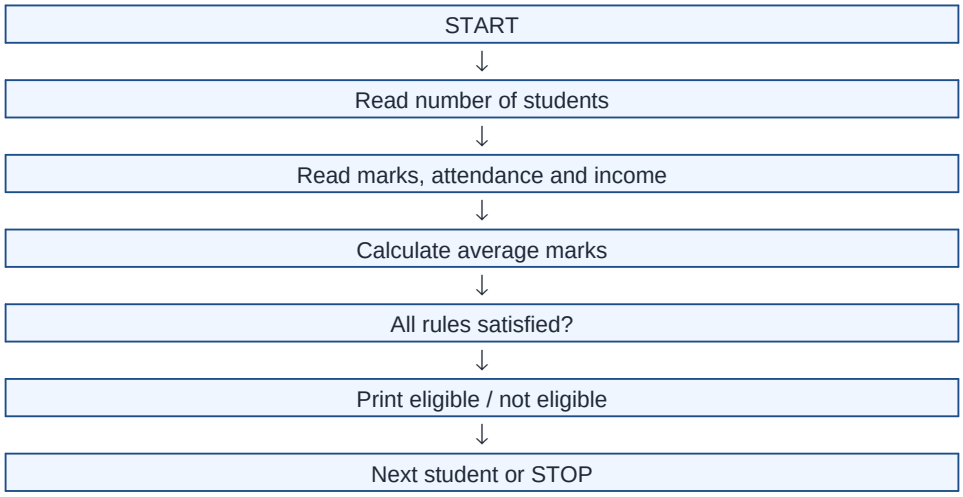
2. For each student, read name, three marks, attendance, and income.

3. Calculate average = total marks / 3.

4. If every condition is met, print eligible; otherwise print not eligible.

5. Repeat for all students and stop.

Flowchart



Pseudocode

```
START
INPUT n
FOR each student
  INPUT name, m1, m2, m3, attendance, income
  average = (m1 + m2 + m3) / 3
  IF average >= 75 AND m1,m2,m3 >= 60 AND attendance >= 75 AND income <= 250000
    PRINT eligible
  ELSE PRINT not eligible
END FOR
STOP
```

Python Source Code

```
n = int(input("Enter number of students: "))

for i in range(n):
    print("\\nStudent", i + 1)
```

```
name = input("Enter name: ")
m1 = float(input("Subject 1 marks: "))
m2 = float(input("Subject 2 marks: "))
m3 = float(input("Subject 3 marks: "))
attendance = float(input("Attendance percentage: "))
income = float(input("Annual family income: "))

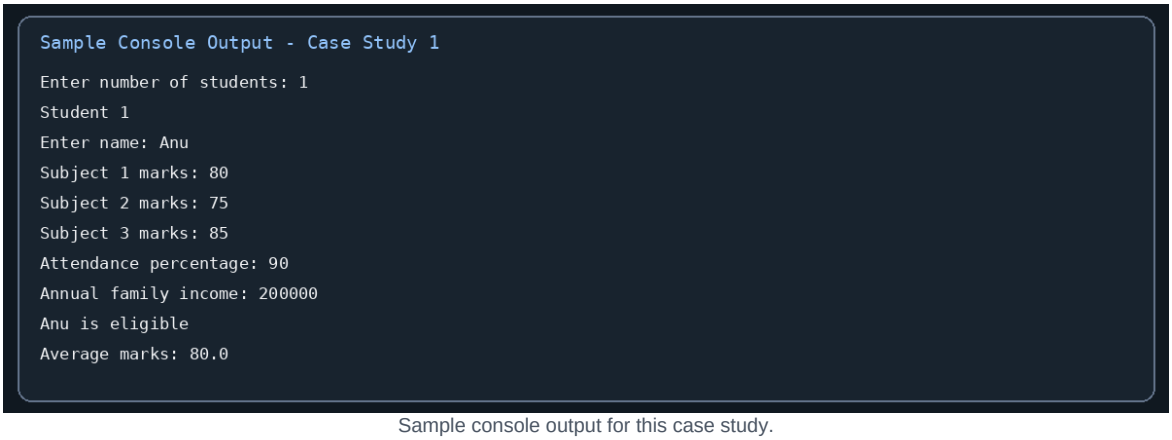
average = (m1 + m2 + m3) / 3

if (average >= 75 and m1 >= 60 and m2 >= 60
        and m3 >= 60 and attendance >= 75
        and income <= 250000):
    print(name, "is eligible")
    print("Average marks:", round(average, 2))
else:
    print(name, "is not eligible")
```

Test Cases

Inputs	Expected result
80, 75, 85; attendance 90; income 200000	Eligible; average 80
55, 90, 90; attendance 90; income 200000	Not eligible: subject below 60
80, 80, 80; attendance 70; income 200000	Not eligible: attendance
80, 80, 80; attendance 90; income 300000	Not eligible: income

Output Screenshot



Sample console output for this case study.

Conclusion

The program applies arithmetic and conditional statements to evaluate scholarship eligibility. More subjects and ranking rules can be added later.

2. Attendance Recovery Computation

Problem Statement

Calculate the minimum number of consecutive classes a student must attend to reach a required attendance percentage.

Objectives

• Validate attendance data.

• Calculate current attendance.

• Simulate future classes using a loop.

• Report extra classes and final attendance.

Assumptions

• The student attends every future class.

• Held classes must be positive; attended classes must be between 0 and held classes.

• Target attendance must be between 0% and 100%.

Algorithm

1. Read classes held, classes attended, and target percentage.

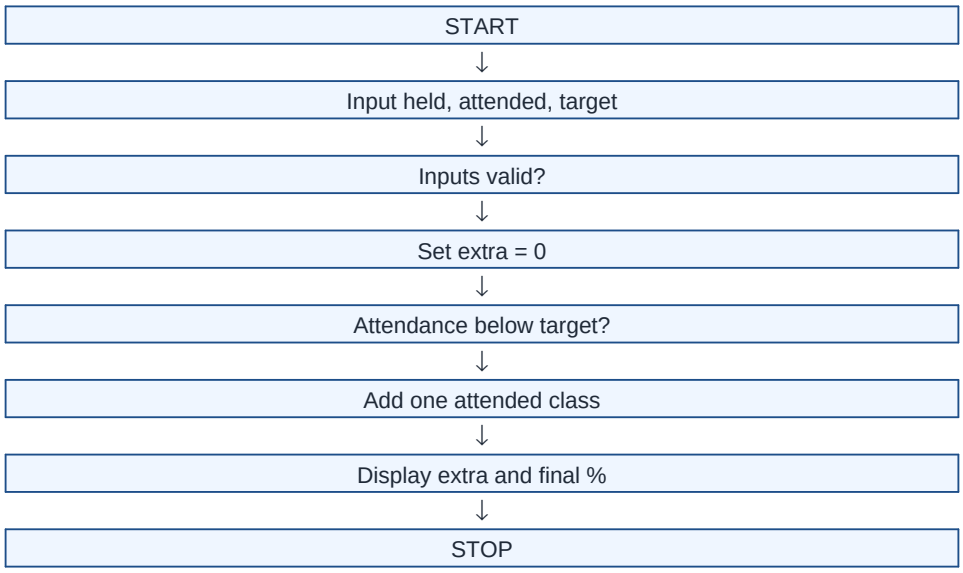
2. Validate the inputs.

3. Set extra = 0.

4. While attendance is below the target, increase held, attended, and extra by one.

5. Display extra classes and final percentage.

Flowchart



Pseudocode

```
START
INPUT held, attended, target
IF held <= 0 OR attended < 0 OR attended > held OR target < 0 OR target > 100: PRINT invalid
ELSE extra = 0
  WHILE attended * 100 / held < target
    attended = attended + 1; held = held + 1; extra = extra + 1
  PRINT extra and final attendance
STOP
```

Python Source Code

```
held = int(input("Enter classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Required attendance percentage: "))

if (held <= 0 or attended < 0 or attended > held
        or target < 0 or target > 100):
    print("Invalid attendance details")
else:
```

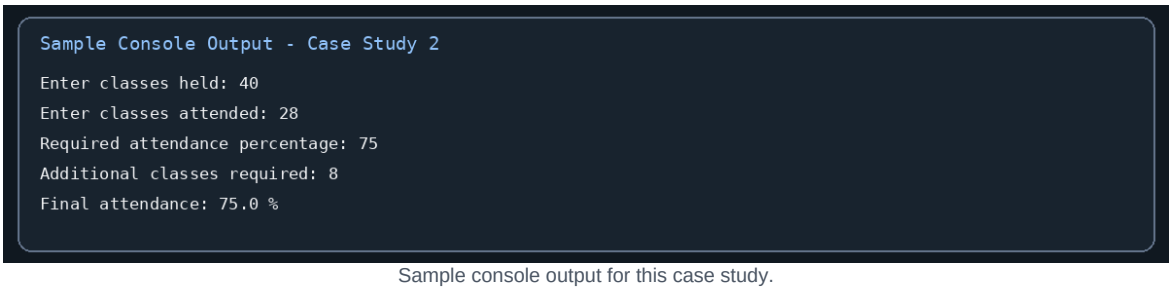
```
extra = 0
while attended * 100 / held < target:
    attended = attended + 1
    held = held + 1
    extra = extra + 1

print("Additional classes required:", extra)
print("Final attendance:", round(attended * 100 / held, 2), "%")
```

Test Cases

Held / attended / target	Expected result
40 / 28 / 75%	8 extra classes; final 75%
30 / 24 / 75%	0 extra classes; already at 80%
0 / 0 / 75%	Invalid attendance details
20 / 10 / 101%	Invalid attendance details

Output Screenshot



Sample console output for this case study.

Conclusion

A loop simulates future attendance until the target is reached. Validation avoids impossible input values.

3. ATM Transaction Sequence Simulation

Problem Statement

Process deposits and withdrawals, update the account balance, reject invalid transactions, and display the final balance.

Objectives

• Maintain the balance through multiple transactions.

• Use a loop for repeated transactions.

• Validate amounts and transaction choices.

• Enforce the withdrawal limit.

Assumptions

• Opening balance cannot be negative.

• Transaction amounts must be positive.

• Maximum withdrawal per transaction is ■10,000.

• No bank fees are included.

Algorithm

1. Read opening balance and number of transactions.

2. For each transaction, read choice and amount.

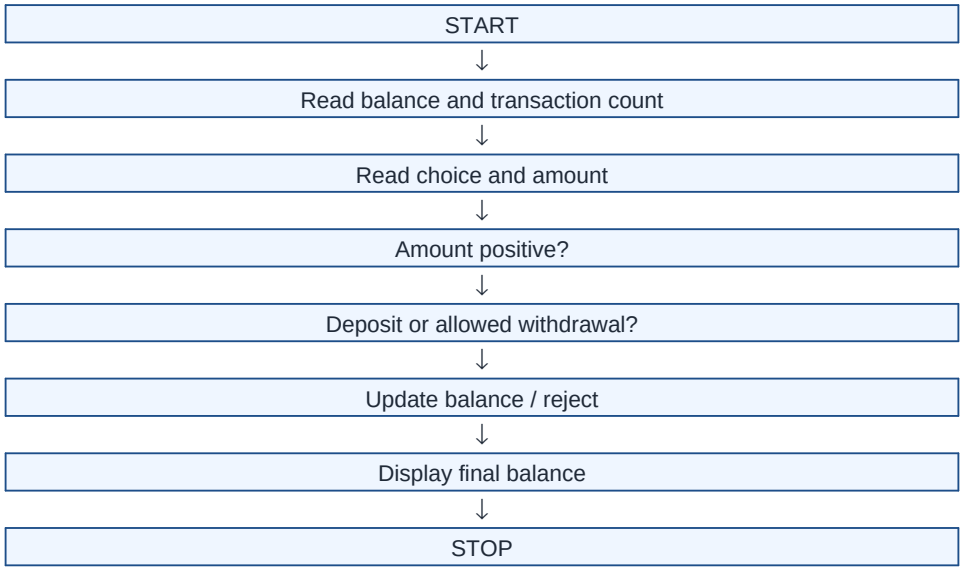
3. Reject non-positive amounts.

4. For deposit, add the amount.

5. For withdrawal, subtract only if amount is within balance and limit.

6. Display final balance.

Flowchart



Pseudocode

```
START
INPUT balance, n
FOR each transaction
  INPUT choice, amount
  IF amount <= 0: PRINT invalid amount
  ELSE IF choice == 1: balance = balance + amount
  ELSE IF choice == 2 AND amount <= balance AND amount <= 10000: balance = balance - amount
  ELSE IF choice == 2: PRINT withdrawal rejected
  ELSE PRINT invalid choice
PRINT balance
STOP
```

Python Source Code

```
balance = float(input("Enter opening balance: "))
```

```
n = int(input("Enter number of transactions: "))

if balance < 0 or n < 0:
    print("Invalid starting details")
else:
    for i in range(n):
        print("\\n1. Deposit")
        print("2. Withdraw")
        choice = int(input("Enter choice: "))
        amount = float(input("Enter amount: "))

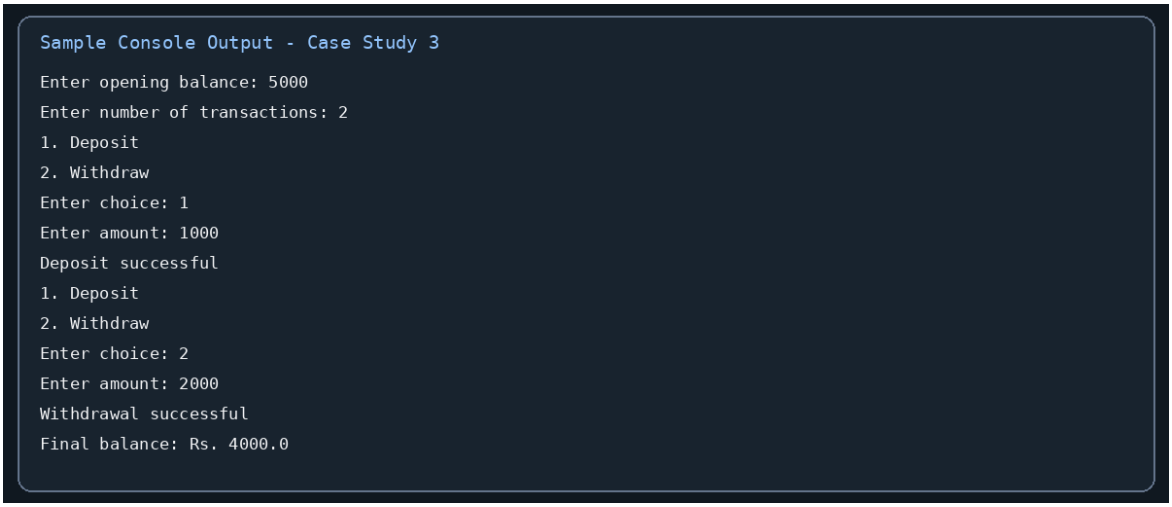
        if amount <= 0:
            print("Invalid amount")
        elif choice == 1:
            balance = balance + amount
            print("Deposit successful")
        elif choice == 2:
            if amount <= balance and amount <= 10000:
                balance = balance - amount
                print("Withdrawal successful")
            else:
                print("Insufficient balance or limit exceeded")
        else:
            print("Invalid choice")

    print("Final balance: Rs.", round(balance, 2))
```

Test Cases

Input	Expected result
Opening ■5000; deposit ■1000	Balance becomes ■6000
Opening ■5000; withdraw ■2000	Balance becomes ■3000
Opening ■5000; withdraw ■6000	Rejected: insufficient balance
Withdraw ■12000	Rejected: exceeds ■10000 limit

Output Screenshot



Sample console output for this case study.

Conclusion

The program demonstrates loops, conditions, and balance updates. It is a classroom simulation, not a real banking system.

4. Smart Parking Fee Management

Problem Statement

Calculate a parking fee based on vehicle type and parking duration, rounding partial hours up and applying a maximum fee.

Objectives

• Read parking duration and vehicle type.

• Select the correct hourly rate.

• Round up partial hours.

• Apply a maximum fee and show the result.

Assumptions

• Car rate is ■30 per hour; bike rate is ■15 per hour.

• Any partial hour is charged as a full hour.

• Maximum fee is ■300.

• Overnight penalties are not included.

Algorithm

1. Read duration and vehicle type.

2. Reject duration ≤ 0.

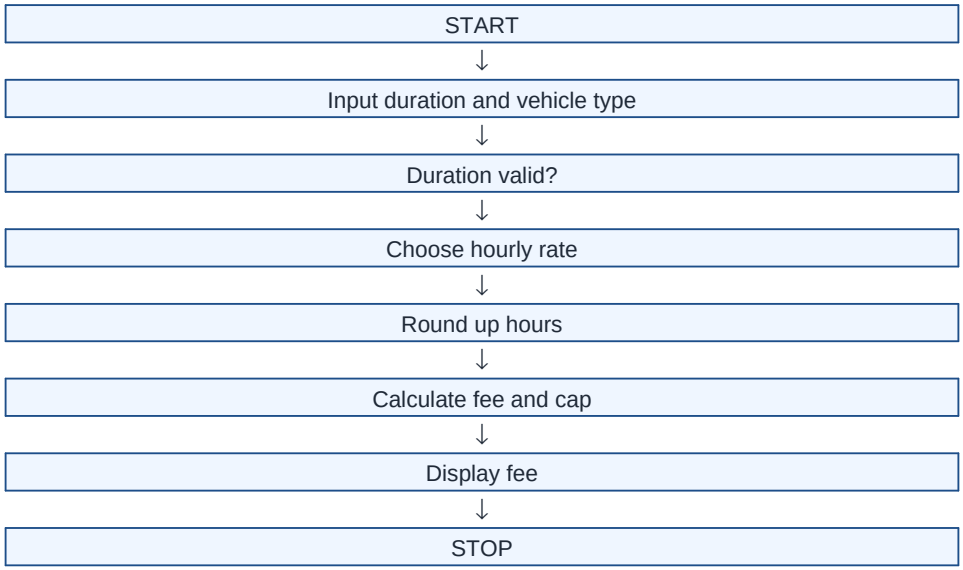
3. Choose rate for car or bike; reject other vehicle types.

4. Round up the duration to whole hours.

5. Calculate fee and cap at ■300.

6. Print hours charged and fee.

Flowchart



Pseudocode

```
START
INPUT hours, vehicle
IF hours <= 0: PRINT invalid duration
ELSE choose rate: car=30, bike=15, otherwise invalid
charged_hours = integer part of hours
IF hours > charged_hours: charged_hours = charged_hours + 1
fee = charged_hours * rate
IF fee > 300: fee = 300
PRINT fee
STOP
```

Python Source Code

```
hours = float(input("Enter parking duration in hours: "))
vehicle = input("Enter vehicle type (car/bike): ").lower()
```

```
if hours <= 0:
    print("Invalid parking duration")
else:
    if vehicle == "car":
        rate = 30
    elif vehicle == "bike":
        rate = 15
    else:
        rate = 0

    if rate == 0:
        print("Invalid vehicle type")
    else:
        charged_hours = int(hours)
        if hours > charged_hours:
            charged_hours = charged_hours + 1

        fee = charged_hours * rate
        if fee > 300:
            fee = 300

        print("Parking hours charged:", charged_hours)
        print("Parking fee: Rs.", fee)
```

Test Cases

Duration / vehicle	Expected result
2 hours / car	2 hours; ■60
2.5 hours / bike	3 hours; ■45
12 hours / car	Fee capped at ■300
-1 hour / car	Invalid duration
2 hours / truck	Invalid vehicle

Output Screenshot



Sample console output for this case study.

Conclusion

Conditions and simple calculations apply parking rates, round partial hours upward, and enforce the maximum fee.

5. Student Performance Trend Analysis

Problem Statement

Store marks from multiple examinations, calculate the average, compare consecutive results, and classify the latest performance.

Objectives

• Store marks in a list.

• Calculate average marks.

• Identify improvement, decline, or no change.

• Classify the latest mark.

Assumptions

• Marks are out of 100.

• At least one exam must be entered.

• Latest mark ≥75 is Excellent; ≥50 and <75 is Average; below 50 Needs improvement.

Algorithm

1. Read number of examinations and ensure it is positive.

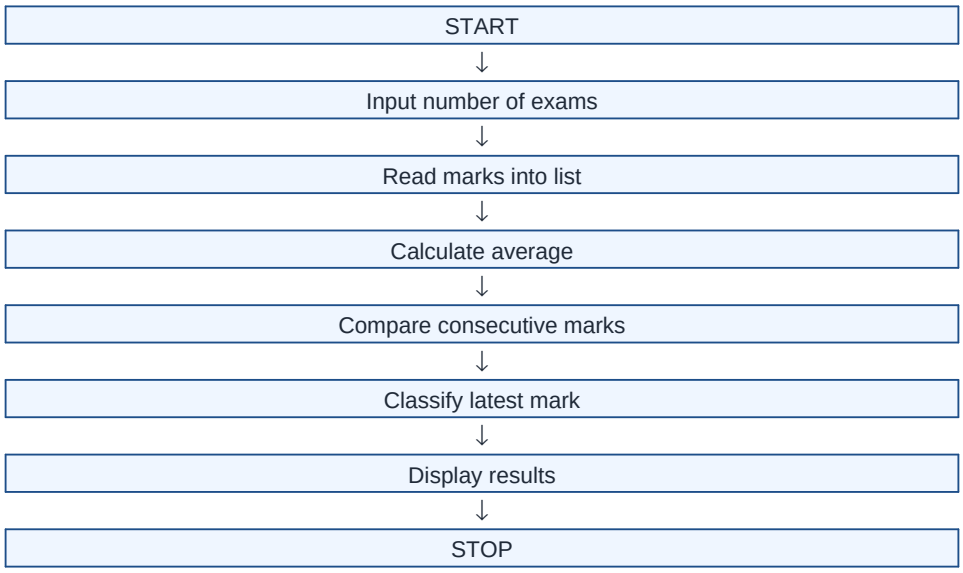
2. Read marks into a list.

3. Calculate and print average.

4. Compare each mark with the previous mark.

5. Classify the latest mark and stop.

Flowchart



Pseudocode

```
START
INPUT n
IF n <= 0: PRINT enter at least one exam
ELSE read n marks into list
PRINT sum(marks) / n
FOR i from 1 to n-1: compare marks[i] with marks[i-1]
CLASSIFY last mark
STOP
```

Python Source Code

```
n = int(input("Enter number of examinations: "))

if n <= 0:
    print("Enter at least one examination")
else:
    marks = []
    for i in range(n):
        mark = float(input("Enter marks out of 100: "))
```

```
    marks.append(mark)

print("Marks:", marks)
print("Average marks:", round(sum(marks) / n, 2))

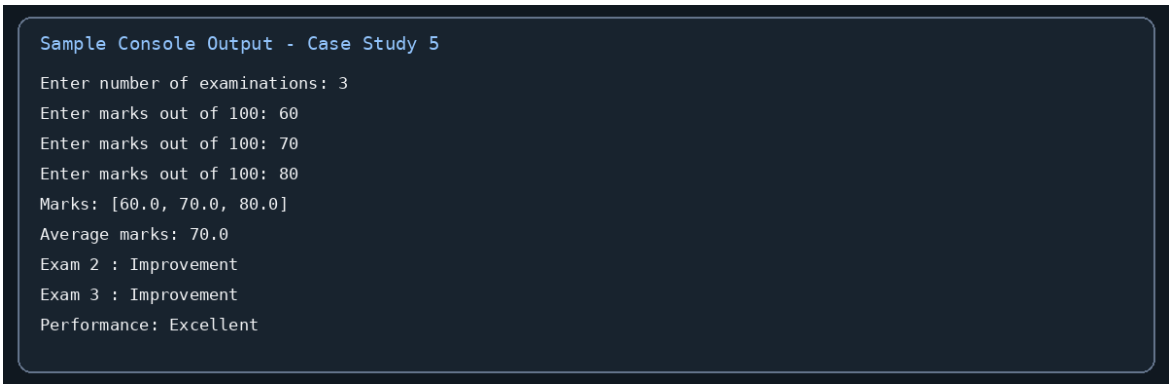
for i in range(1, n):
    if marks[i] > marks[i - 1]:
        print("Exam", i + 1, ": Improvement")
    elif marks[i] < marks[i - 1]:
        print("Exam", i + 1, ": Decline")
    else:
        print("Exam", i + 1, ": No change")

if marks[-1] >= 75:
    print("Performance: Excellent")
elif marks[-1] >= 50:
    print("Performance: Average")
else:
    print("Performance: Needs improvement")
```

Test Cases

Marks	Expected result
60, 70, 80	Average 70; improvement twice; latest Excellent
80, 70, 65	Average 71.67; decline twice; latest Average
40, 40	No change; Needs improvement
n = 0	Enter at least one examination

Output Screenshot



Sample console output for this case study.

Conclusion

Lists store exam results and make comparison easy. The output shows both average performance and the trend between exams.

6. Fuel Consumption and Route Comparison

Problem Statement

Calculate fuel required and estimated journey cost for several vehicles, including reserve fuel, and identify the lowest-cost valid option.

Objectives

• Read distance and fuel price.

• Calculate fuel required from mileage and reserve.

• Calculate each vehicle's journey cost.

• Select the lowest-cost valid vehicle.

Assumptions

• Distance and fuel price must be positive.

• Mileage must be greater than zero; reserve fuel cannot be negative.

• Fuel = distance / mileage + reserve fuel.

• Only valid vehicle records are compared.

Algorithm

1. Read distance, fuel price, and number of vehicles.

2. For each vehicle, read name, mileage, and reserve fuel.

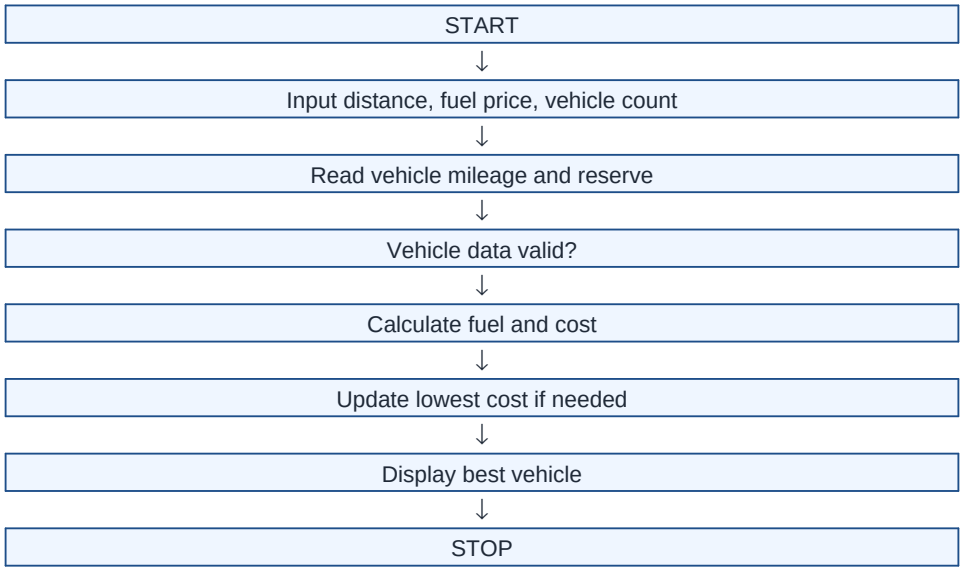
3. Reject invalid vehicle details.

4. Calculate fuel and cost for valid details.

5. Track the vehicle with the lowest cost.

6. Print the best vehicle and cost, or report no valid vehicles.

Flowchart



Pseudocode

```
START
INPUT distance, fuel_price, n
best_name = empty
FOR each vehicle: INPUT name, mileage, reserve
IF mileage <= 0 OR reserve < 0: PRINT invalid
ELSE fuel = distance / mileage + reserve; cost = fuel * fuel_price
IF best_name empty OR cost < best_cost: save this vehicle and cost
PRINT lowest-cost valid vehicle
STOP
```

Python Source Code

```
distance = float(input("Enter journey distance in km: "))
```

```
fuel_price = float(input("Enter fuel price per litre: Rs. "))
n = int(input("Enter number of vehicles: "))

if distance <= 0 or fuel_price <= 0 or n <= 0:
    print("Invalid journey details")
else:
    best_name = ""
    best_cost = 0

    for i in range(n):
        name = input("Enter vehicle name: ")
        mileage = float(input("Mileage in km per litre: "))
        reserve = float(input("Reserve fuel in litres: "))

        if mileage <= 0 or reserve < 0:
            print("Invalid vehicle details")
        else:
            fuel = distance / mileage + reserve
            cost = fuel * fuel_price
            print("Fuel required:", round(fuel, 2), "litres")
            print("Journey cost: Rs.", round(cost, 2))

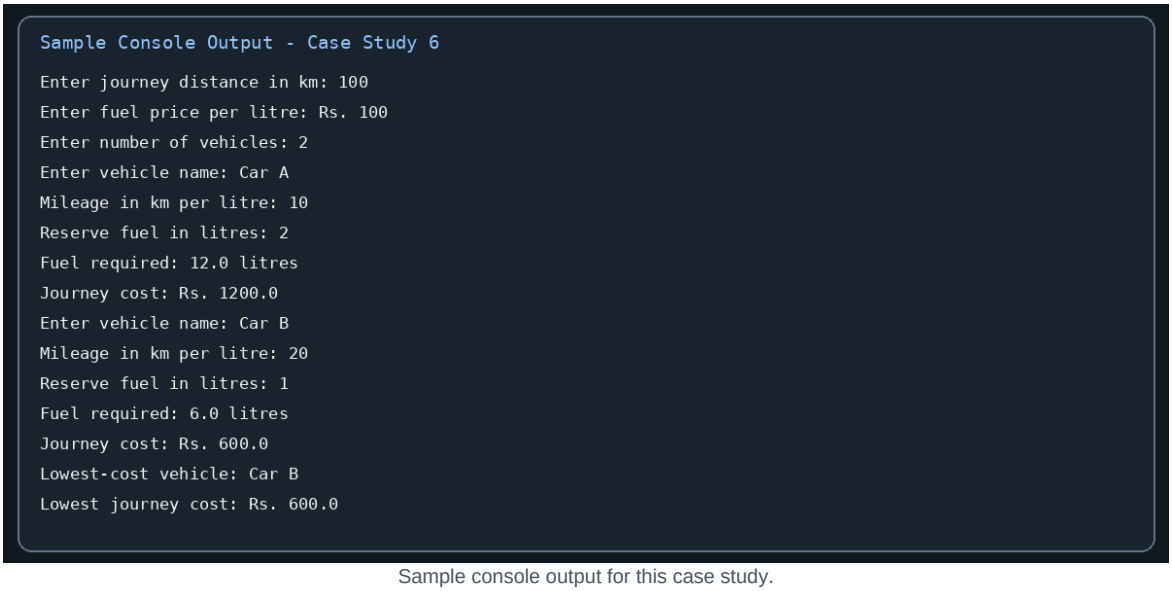
            if best_name == "" or cost < best_cost:
                best_cost = cost
                best_name = name

    if best_name == "":
        print("No valid vehicle details")
    else:
        print("Lowest-cost vehicle:", best_name)
        print("Lowest journey cost: Rs.", round(best_cost, 2))
```

Test Cases

Distance / fuel price / vehicle	Expected result
100 km / ■100/L / mileage 10, reserve 2	12 L; ■1200
100 km / ■100/L / mileage 20, reserve 1	6 L; ■600; lowest if compared with previous
100 km / ■100/L / mileage 0	Invalid vehicle details
Distance 0	Invalid journey details

Output Screenshot



Sample console output for this case study.

Conclusion

The program compares fuel costs using mileage and reserve fuel. Actual costs can vary with traffic, terrain, and vehicle condition.

7. Multi-Product Shopping Budget Constraint

Problem Statement

Calculate the total cost of several products and determine whether the shopping basket stays within a fixed budget.

Objectives

• Read product name, price, and quantity.

• Calculate each product subtotal.

• Add subtotals to get total cost.

• Compare total cost with budget.

Assumptions

• Prices and quantities cannot be negative.

• No tax or discount is included.

• The program reports the amount over budget but does not automatically choose products to remove.

Algorithm

1. Read budget and product count.

2. Set total to zero.

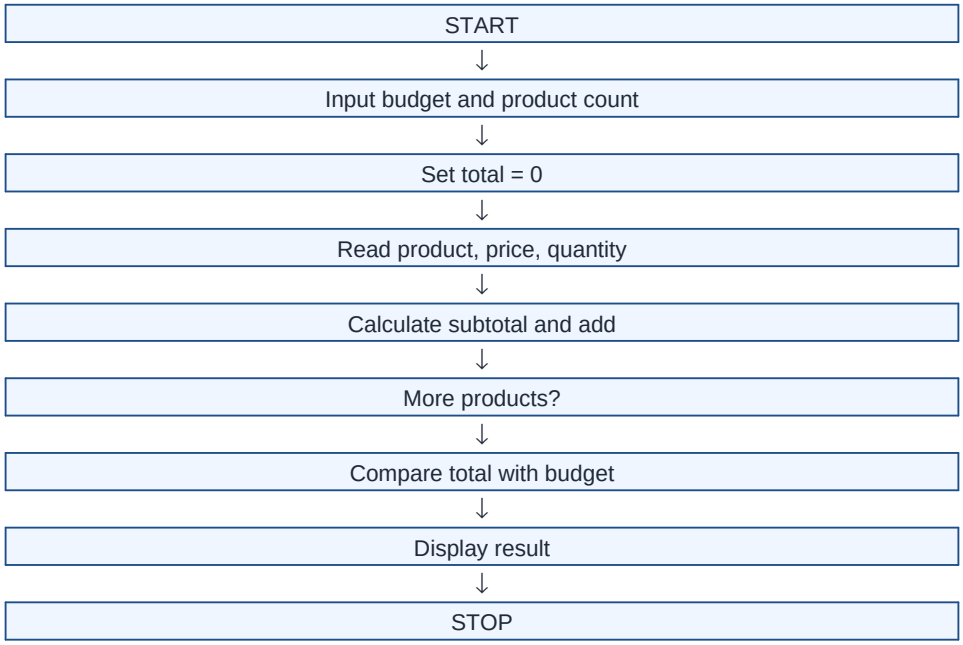
3. For each product, read name, price, and quantity.

4. Calculate subtotal and add it to total.

5. Display the total.

6. If total ≤ budget, print within budget; otherwise print excess.

Flowchart



Pseudocode

```
START
INPUT budget, n
total = 0
FOR each product: INPUT name, price, quantity
IF price < 0 OR quantity < 0: PRINT invalid
ELSE subtotal = price * quantity; total = total + subtotal
PRINT total
IF total <= budget: PRINT within budget
ELSE PRINT total - budget excess
STOP
```

Python Source Code

```
budget = float(input("Enter shopping budget: Rs. "))
n = int(input("Enter number of products: "))

if budget < 0 or n < 0:
    print("Invalid budget or product count")
else:
    total = 0
    for i in range(n):
        name = input("Enter product name: ")
        price = float(input("Enter price: Rs. "))
        quantity = int(input("Enter quantity: "))

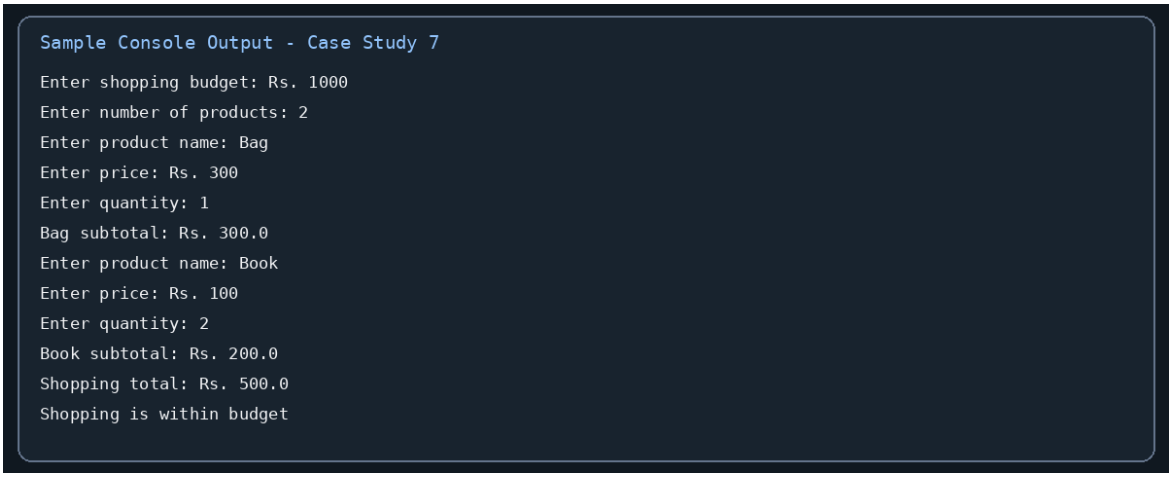
        if price < 0 or quantity < 0:
            print("Invalid price or quantity")
        else:
            amount = price * quantity
            total = total + amount
            print(name, "subtotal: Rs.", round(amount, 2))

    print("Shopping total: Rs.", round(total, 2))
    if total <= budget:
        print("Shopping is within budget")
    else:
        print("Budget exceeded by Rs.", round(total - budget, 2))
```

Test Cases

Budget and products	Expected result
■1000; bag ■300×1, book ■100×2	Total ■500; within budget
■500; shoes ■700×1	Total ■700; excess ■200
■500; pens ■10×5	Total ■50; within budget
Negative price	Invalid price message

Output Screenshot



Sample console output for this case study.

Conclusion

The program practices loops, product-wise calculations, and budget checks. Discounts and tax rules can be added later.

8. Advanced Numeric Pattern Generator

Problem Statement

Print a symmetric numeric pattern using nested loops, and reject invalid row counts.

Objectives

• Use nested loops to print a pattern.

• Validate the requested number of rows.

• Understand loop ranges and output formatting.

Assumptions

• The row count must be a positive integer.

• The pattern increases from 1 to n, then decreases from n-1 to 1.

• This is a simple symmetric numeric pattern, not a hollow diamond.

Algorithm

1. Read row count n.

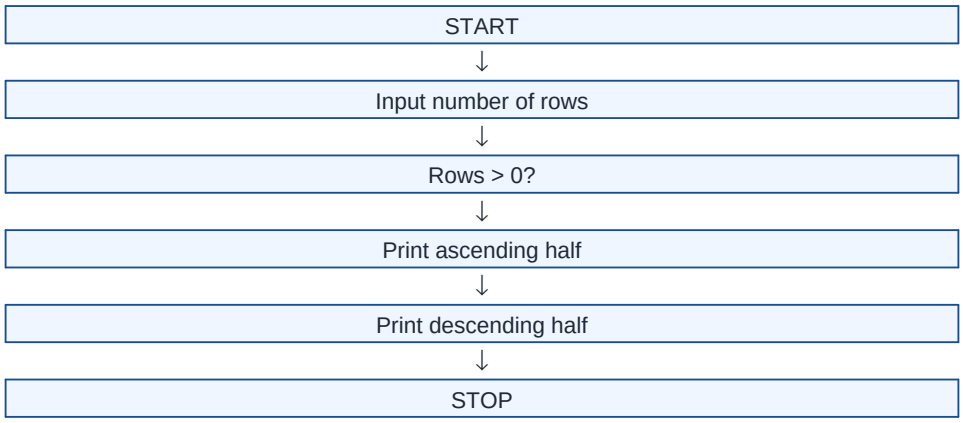
2. If n ≤ 0, display an error.

3. Use nested loops to print the ascending half.

4. Use nested loops to print the descending half.

5. Stop.

Flowchart



Pseudocode

```
START
INPUT n
IF n <= 0: PRINT positive number required
ELSE FOR i = 1 TO n
  FOR j = 1 TO i: PRINT j without newline
  PRINT newline
FOR i = n-1 DOWNTO 1: repeat inner loop
STOP
```

Python Source Code

```
n = int(input("Enter number of rows: "))

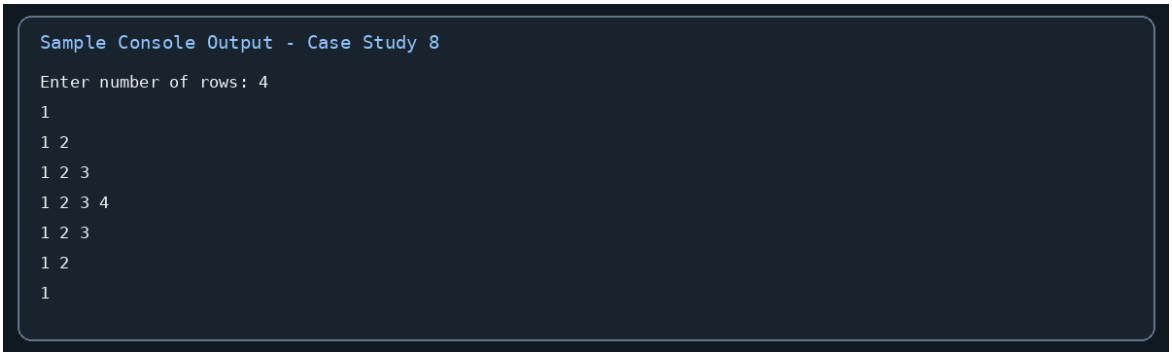
if n <= 0:
    print("Enter a positive number")
else:
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()

    for i in range(n - 1, 0, -1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

Test Cases

Input	Expected result
1	Prints only 1
3	1; 1 2; 1 2 3; 1 2; 1
4	Pattern grows to 1 2 3 4 then shrinks
0	Prints positive number error

Output Screenshot



Sample console output for this case study.

Conclusion

Nested loops control the rows and numbers printed. This exercise builds understanding of iteration and pattern generation.

9. Password Policy Validation Engine

Problem Statement

Check whether a password meets minimum length and contains uppercase, lowercase, and numeric characters; report any missing rules.

Objectives

• Process the password one character at a time.

• Check character categories.

• Check minimum length.

• Explain why a password is invalid.

Assumptions

• At least eight characters are required.

• At least one uppercase letter, lowercase letter, and digit are required.

• This simple classroom check is not a complete real-world security system.

Algorithm

1. Read password.

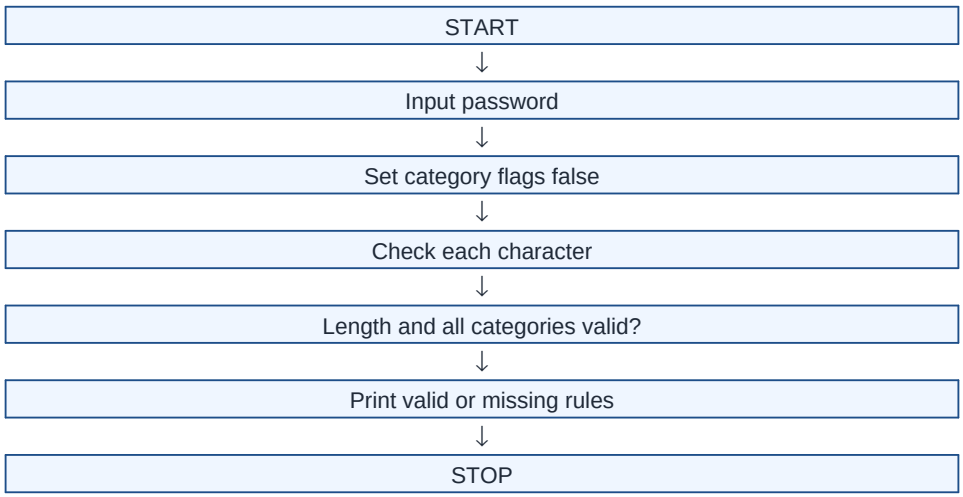
2. Set uppercase, lowercase, and digit flags to false.

3. Inspect each character and update flags.

4. Check length and all flags.

5. Print valid, or print invalid and every missing condition.

Flowchart



Pseudocode

```
START
INPUT password
upper = false; lower = false; digit = false
FOR each ch: update the matching flag
IF length >= 8 AND upper AND lower AND digit: PRINT valid
ELSE PRINT invalid and each missing rule
STOP
```

Python Source Code

```
password = input("Enter password: ")

has_upper = False
has_lower = False
has_digit = False

for ch in password:
    if ch >= "A" and ch <= "Z":
        has_upper = True
    elif ch >= "a" and ch <= "z":
        has_lower = True
```

```
    elif ch >= "0" and ch <= "9":
        has_digit = True

if len(password) >= 8 and has_upper and has_lower and has_digit:
    print("Password is valid")
else:
    print("Password is invalid")
    if len(password) < 8:
        print("Use at least 8 characters")
    if not has_upper:
        print("Add an uppercase letter")
    if not has_lower:
        print("Add a lowercase letter")
    if not has_digit:
        print("Add a digit")
```

Test Cases

Password	Expected result
Hello123	Valid
hello123	Uppercase missing
HELLO123	Lowercase missing
HelloWorld	Digit missing
Ab1	Too short and missing categories

Output Screenshot



Sample console output for this case study.

Conclusion

String traversal and Boolean flags make it possible to check simple password rules and report individual failures.

10. Multi-Stage Examination Eligibility System

Problem Statement

Check examination eligibility using attendance, lab completion, internal marks, and fee clearance. Report every unmet requirement.

Objectives

• Read all four eligibility details.

• Check each rule independently.

• Report every unmet requirement.

• Display the final decision.

Assumptions

• Attendance must be ≥75%.

• All lab sessions must be completed.

• Internal marks must be ≥40 out of 100.

• Fees must be fully paid.

• These are illustrative rules; use instructor-specified thresholds if different.

Algorithm

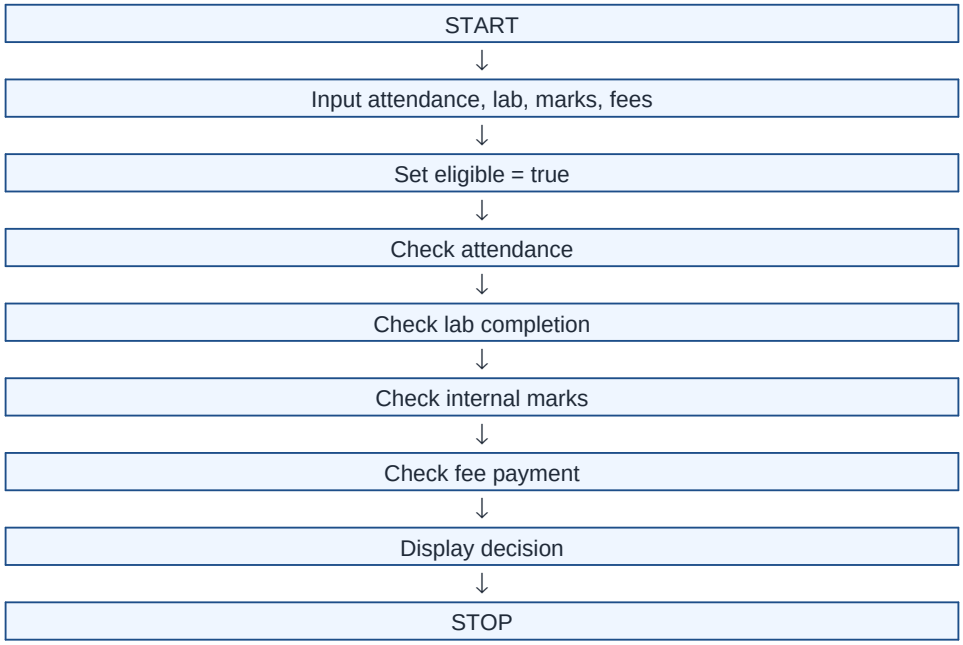
1. Read attendance, lab status, internal marks, and fee status.

2. Set eligible to true.

3. Check each condition; print the reason and set eligible false for every failed condition.

4. Print eligible if no condition failed; otherwise print not eligible.

Flowchart



Pseudocode

```
START
INPUT attendance, lab, internal, fees
eligible = true
IF attendance < 75: PRINT reason; eligible = false
IF lab != yes: PRINT reason; eligible = false
IF internal < 40: PRINT reason; eligible = false
IF fees != yes: PRINT reason; eligible = false
IF eligible: PRINT eligible ELSE PRINT not eligible
STOP
```

Python Source Code

```
attendance = float(input("Enter attendance percentage: "))
```

```
lab = input("Are all lab sessions completed? (yes/no): ").lower()
internal = float(input("Enter internal marks out of 100: "))
fees = input("Are fees fully paid? (yes/no): ").lower()

eligible = True

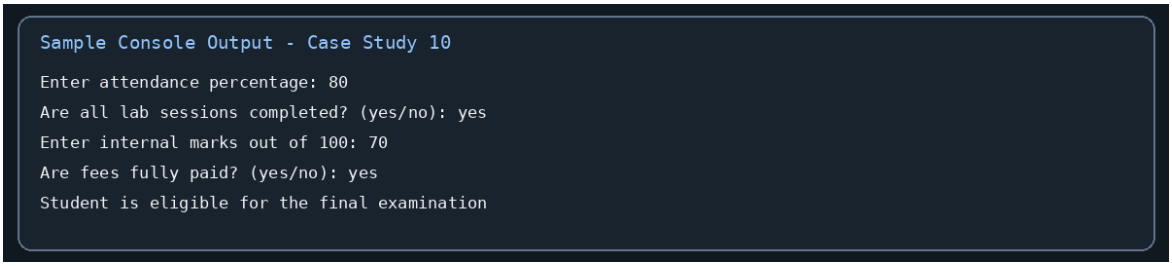
if attendance < 75:
    print("Not eligible: Attendance is below 75%")
    eligible = False
if lab != "yes":
    print("Not eligible: Lab work is incomplete")
    eligible = False
if internal < 40:
    print("Not eligible: Internal marks are below 40")
    eligible = False
if fees != "yes":
    print("Not eligible: Fees are not fully paid")
    eligible = False

if eligible:
    print("Student is eligible for the final examination")
else:
    print("Student is not eligible for the final examination")
```

Test Cases

Attendance / labs / marks / fees	Expected result
80 / yes / 70 / yes	Eligible
70 / yes / 70 / yes	Not eligible: attendance
80 / no / 35 / no	Not eligible: lab, marks, and fees
75 / yes / 40 / yes	Eligible at boundaries

Output Screenshot



Sample console output for this case study.

Conclusion

The program checks multiple requirements independently and explains each failed condition, making the decision easy to understand.